

# Ficha da Etapa 1 — Projeto de Solução Distribuída

Disciplina: 070080 Sistemas Distribuídos e Paralelos · Oficina: 08/09 · Turma: CC6MA · Professor: Fábio Rocha de Araújo · Etapa 1 completa: 22/09

## A. Equipe e problema

| # | Nome          | Matrícula |
|---|---------------|-----------|
| 1 | Yuri Aguiar   | 24070309  |
| 2 | Pedro Paulo   | 24070313  |
| 3 | João Rath     | 24070338  |
| 4 | Fábio Gabriel | 21070209  |

**Problema escolhido:** triagem automática de editais de compra pública. A aplicação lê um lote de editais em texto, pontua cada um contra uma taxonomia de termos ligados a software de gestão e arrecadação pública, classifica por linha de produto e calcula uma assinatura MinHash do objeto para identificar o mesmo edital republicado por municípios diferentes. A saída é um relatório com ranking, distribuição por linha, frequência de termos e contagem de republicações.

| Condição                                                                   | Resposta da equipe                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| O trabalho se divide em partes independentes? Em que unidade?              | Sim. A unidade é <b>um edital</b> . O processamento de um edital não lê nem escreve nada de outro: as funções de <code>src/nucLeo.py</code> são puras e o módulo não tem estado global. A dependência só aparece na agregação dos resultados, e a agregação é associativa e comutativa.                                                                                                                                               |
| Qual é o volume da entrada, e quanto tempo a versão sequencial deve levar? | 15000 editais de cerca de 10 KiB cada, aproximadamente 150 MiB de texto. O volume não foi arbitrado: <code>src/calibrar.py</code> mede o custo por documento na própria instância e calcula quantos documentos são precisos para o alvo de <b>4 minutos</b> de execução sequencial.                                                                                                                                                   |
| Como se verifica que o resultado está correto?                             | O relatório final é canonizado — a ordenação depende só do conteúdo, nunca da ordem de chegada — e reduzido a um <b>SHA-256</b> . <code>src/verificar.py</code> roda a versão sequencial uma vez e a paralela várias vezes sobre a mesma entrada e compara as impressões digitais. São duas verificações distintas: <i>equivalência</i> (paralela = sequencial) e <i>estabilidade</i> (todas as execuções paralelas iguais entre si). |
| Que estado é escrito por mais de um fluxo?                                 | O agregador global, em memória compartilhada por um <code>multiprocessing.Manager</code> : contador de documentos e de tokens, soma de pontuação, dicionário de frequência dos termos da taxonomia, contagem por linha de produto, ranking dos 25 melhores, mapa de assinaturas e relógio lógico.                                                                                                                                     |

## B. Estratégia de paralelização

| Decisão                               | Escolha             | Justificativa                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Paralelismo de dados ou de tarefas    | <b>Dados</b>        | A mesma sequência de operações se aplica a documentos diferentes. Não existem estágios heterogêneos que pudessem correr simultaneamente, então paralelismo de tarefas não se aplica.                                                                                                                                                                                                                                                                                                                                                      |
| Processo ou thread                    | <b>Processo</b>     | O trabalho é <b>limitado por processador</b> , não por espera: a leitura do arquivo responde por menos de 0,5 % do tempo total e o restante é normalização Unicode, expressão regular, hashing e aritmética de inteiros em Python puro. Em CPython o GIL impede que duas threads executem bytecode ao mesmo tempo, de modo que a versão com threads mediria speedup próximo de 1 e a análise concluiria o oposto do correto. A aplicação tem o modo <code>--motor thread</code> para que essa afirmação seja medida, e não apenas citada. |
| Quantos trabalhadores em paralelo     | 1, 2, 4 e 8         | A instância <code>c6i.2xlarge</code> tem 8 vCPU sobre 4 núcleos físicos. Mede-se a curva inteira para localizar onde o ganho satura: espera-se escala próxima da linear até $n = 4$ e ganho marginal pequeno de 4 para 8, quando os trabalhadores passam a dividir núcleos físicos.                                                                                                                                                                                                                                                       |
| Como o trabalho é dividido entre eles | Fila comum de lotes | O corpus é cortado em lotes de 150 documentos, todos postos numa única <code>multiprocessing.Queue</code> . Cada trabalhador puxa o próximo lote quando termina o anterior: balanceamento dinâmico. Divisão fixa em $n$ fatias deixaria o tempo total refem da fatia mais cara.                                                                                                                                                                                                                                                           |

C. Sincronização

| Item                                                                | Resposta da equipe                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Que estrutura é compartilhada e escrita por mais de um fluxo        | O agregador global (estado, assinaturas e eventos, todos proxies de Manager), escrito por todos os trabalhadores ao fim de cada lote.                                                                                                                                                                                                                                                                                                                                                          |
| Qual é o menor trecho que precisa ser indivisível (a seção crítica) | A função <code>fundir_no_global</code> , em <code>src/paralelo.py</code> : a fusão de <b>um agregado parcial</b> no agregado global. O processamento dos 150 documentos do lote acontece fora da trava, num parcial local ao processo. Dentro da trava ocorrem duas operações que não toleram entrelaçamento: a leitura-modificação-escrita dos contadores ( <code>estado["documentos"] = estado["documentos"] + k</code> ) e o <code>verificar-entao-agir</code> sobre o mapa de assinaturas. |
| Que primitiva vai protegê-la: lock, semáforo ou monitor             | <b>multiprocessing.Lock</b> . O recurso admite um escritor por vez e não há condição de espera a sinalizar nem contagem de permissões a administrar, o que dispensa semáforo contado e monitor. A trava é liberada em <code>finally</code> , para que uma exceção dentro da seção crítica não trave os demais processos.                                                                                                                                                                       |
| Como a equipe vai demonstrar que o resultado ficou estável          | Ao vivo, com <code>src/verificar.py</code> : a mesma entrada, oito execuções paralelas, oito SHA-256 idênticos e iguais ao da versão sequencial. Em seguida a mesma verificação com <code>--modo nenhuma</code> , que remove a trava: aí aparecem impressões digitais diferentes a cada execução e contagens de documentos menores que o total, porque atualizações se perdem. A corrida é exibida, não descrita.                                                                              |

**Registro de evento e carimbo lógico.** Há troca de mensagem: a fila de lotes e o canal até o processo do Manager. Como os processos não compartilham relógio físico, a ordem das fusões é registrada por um **relógio lógico de Lamport**: cada trabalhador mantém um contador local, incrementa-o ao receber um lote e, ao fundir, adianta-o para `max(local, global) + 1`, grava esse valor no estado compartilhado e registra o evento. Cada registro contém:

| Campo          | Conteúdo                                           |
|----------------|----------------------------------------------------|
| quem           | identificador do trabalhador (w0 a w7)             |
| o quê          | <code>fusao_no_agregador_global</code>             |
| carimbo lógico | valor do relógio de Lamport após o avanço          |
| sobre o quê    | número do lote e quantidade de documentos fundidos |

O registro completo é gravado em `resultados/eventos.json`, ordenado por carimbo, e permite reconstruir uma ordenação consistente das fusões sem relógio comum.

D. Medição de desempenho

| Item                                                     | Resposta da equipe                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Máquina em que as duas versões serão medidas             | Instância única <code>c6i.2xlarge</code> (AWS EC2), região <code>sa-east-1</code> (São Paulo), zona <code>sa-east-1a</code> . As duas versões rodam na mesma instância, na mesma sessão, sem outra carga.                                                                                                                                                                                                                  |
| Número de núcleos disponíveis nela                       | 8 vCPU sobre 4 núcleos físicos (SMT ativo). A distinção importa: a lei de Amdahl supõe n processadores independentes, e dois vCPU sobre o mesmo núcleo físico não são independentes para trabalho limitado por processador.                                                                                                                                                                                                |
| Volume de entrada fixado para as medições                | 15000 editais, gerados por <code>src/corpus.py</code> com semente fixa. A mesma semente reproduz byte a byte o mesmo corpus, o que garante que $T(1)$ e $T(n)$ sejam medidos sobre a mesma entrada.                                                                                                                                                                                                                        |
| Como o tempo sequencial será obtido                      | Executando <code>src/sequencial.py</code> , que não tem fila, Manager nem trava, três vezes, tomando a mediana. Não se usa a versão paralela com um trabalhador como $T(1)$ : ela já carrega o custo da infraestrutura de paralelização e inflaria o speedup.                                                                                                                                                              |
| Fração do trabalho que a equipe estima ser paralelizável | $p \approx \mathbf{0,9895}$ (medido por instrumentação). Fica fora do trecho paralelo: a listagem do diretório, a partida dos processos, a redução final e a gravação do relatório. A versão sequencial cronometra separadamente a leitura e o trabalho por documento, de modo que o valor instrumentado entra no relatório de 22/09 junto com a fração serial experimental de Karp-Flatt, $e = (1/S - 1/n) / (1 - 1/n)$ . |
| Speedup previsto pela lei de Amdahl                      | $S = 1 / ((1 - p) + p/n)$ , com $p = 0,9895$ : $\mathbf{n = 2 \rightarrow 1,98; n = 4 \rightarrow 3,88; n = 8 \rightarrow 7,45}$ . A equipe espera ficar <i>abaixo</i> desses tetos, e o relatório vai explicar a diferença pelas três causas que a lei não modela: comunicação entre processos, espera na seção crítica e vCPU que não são núcleos independentes.                                                         |

## E. Arquitetura na nuvem

| Decisão                                           | Escolha da equipe                                                     | Justificativa                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Região, e em quantas zonas de disponibilidade     | sa-east-1 (São Paulo)<br>uma zona (sa-east-1a)                        | Instância isolada em zona única: passa a valer o compromisso de <b>99,5 % (216 min/mes)</b> . Distribuir por duas ou mais zonas elevaria o compromisso para 99,99 % (4,3 min/mes), mas o experimento exige que T(1) e T(n) sejam medidos no mesmo hardware — duas instâncias em zonas diferentes não comparariam a mesma coisa — e a aplicação é um lote sob demanda, sem requisito de disponibilidade contínua. A região de São Paulo foi escolhida por latência até a equipe e porque os dados tratados são de compras públicas brasileiras. |
| Família, tamanho e quantidade de instâncias       | 1 × c6i.2xlarge<br>(8 vCPU, otimizada para computação)                | Família sem créditos de CPU. Instância burstável (t2, t3) é inadequada para medir speedup: ao esgotar os créditos o desempenho cai no meio da medição e T(n) deixa de ser comparável a T(1). Uma só instância, porque o objeto da medida é o ganho por núcleo dentro de uma máquina.                                                                                                                                                                                                                                                           |
| Onde ficam a entrada e a saída, e qual o volume   | EBS gp3 de 30 GiB; cópia no bucket S3 sdp-etapa1-equipe (PREENCHER)   | O corpus (aprox. 150 MiB) é gerado no volume local, para que a leitura não introduza latência de rede dentro do tempo medido. Relatórios, registro de eventos e <code>medicao.json</code> (menos de 5 MiB) são arquivados no S3 com bloqueio de acesso público e criptografia SSE-S3, para que sobrevivam ao encerramento da instância.                                                                                                                                                                                                        |
| Portas abertas, e a origem de cada regra          | 22/tcp ← PREENCHER:<br>/32<br>8000/tcp ← o.o.o.o/o<br>saída: liberada | A porta administrativa <b>não</b> está aberta para o.o.o.o/o porque a porta 22 exposta à internet recebe tentativas automáticas de autenticação de forma contínua; restringir a origem ao endereço da equipe retira a instância dessa superfície sem depender da força da chave. A porta 8000 serve o painel da aplicação e é a única aberta ao público. A saída fica liberada para instalar pacotes e enviar os resultados ao S3.                                                                                                             |
| A instância fica ligada, ou é criada por execução | Ligada durante a janela de medição e apresentação                     | Criar por execução traria hardware subjacente potencialmente diferente entre T(1) e T(n), e o speedup exige a mesma máquina. A instância é encerrada após a apresentação.                                                                                                                                                                                                                                                                                                                                                                      |